

TD de Programmation Orientée Objet

Licence d'Informatique*

Semaine du 16 février 2026

Design pattern Strategy

Dans ce TD nous allons voir comment utiliser le design pattern Strategy pour remplacer les classes `FilterNotNullIterator` et `FilterTransportableIterator` (et bien d'autres) par l'unique classe `FilterArrayIterator`.

Exercice 1 Utilisation de l'interface `Predicate<E>`

1. En Java l'interface `Predicate<E>` est définie de la manière suivante :

```
public interface Predicate<E> {  
    /**  
     * Evaluates this predicate on the given argument.  
     * @param t - the input argument  
     * @return true if the input argument matches the predicate,  
     * otherwise false  
     */  
    boolean test(E t);  
}
```

Définissez deux classes implémentant l'interface `Predicate<E>` : la première `TestNotNull<E>` permettant de tester si un objet est non null, et la seconde `TestTransportable` permettant de tester si un `ObjetZork` est transportable.

2. Utilisez la classe `TestNotNull<E>` dans le code suivant à la place du test `s != null` :

```
String[] tabStr = {null, "aa", null, null, "bb", "cc", null};  
for (String s : tabStr) {  
    if (s != null) {  
        System.out.println(":" + s);  
    }  
}  
System.out.println();
```

3. Utilisez la classe `TestTransportable` dans le code suivant à la place du test `oz.estTransportable()` :

```
ObjetZork oz1 = new ObjetZork("oz1", 10, true),  
    oz2 = new ObjetZork("oz2", 10, true),  
    oz3 = new ObjetZork("oz3", 10, false),  
    oz4 = new ObjetZork("oz4", 10, false),  
    oz5 = new ObjetZork("oz5", 10, true),  
    oz6 = new ObjetZork("oz6", 10, false);  
ObjetZork[] tabOz = {oz1, oz2, oz3, oz4, oz5, oz6};  
for (ObjetZork oz : tabOz) {  
    if (oz.estTransportable()) {  
        System.out.print(":" + oz.getNom());  
    }  
}
```

*Marc Champesme – Département d'Informatique – Institut Galilée

```

}
System.out.println();
// Affichage: :oz1:oz2:oz5

```

Exercice 2 Mise en œuvre du design pattern Strategy.

1. Identifier les similitudes et les différences entre les deux classes précédentes implémentant l'interface `Iterator` : `FilterNotNullIterator` et `FilterTransportableIterator`. À partir de cette analyse identifier comment pourrait être utilisé le design pattern Strategy, pour définir une unique classe générique `FilterArrayIterator` pouvant remplacer les classes `FilterNotNullIterator` et `FilterTransportableIterator`. Définissez cette nouvelle classe.
2. Créez une nouvelle classe implémentant l'interface `Predicate<E>` et utilisez la pour compléter le code suivant afin de pouvoir utiliser un itérateur filtrant les `ObjetZork` de poids supérieur à 10, comme dans l'exemple suivant :

```

ObjetZork oz1 = new ObjetZork("oz1", 5, true),
oz2 = new ObjetZork("oz2", 45, true),
oz3 = new ObjetZork("oz3", 7, false),
oz4 = new ObjetZork("oz4", 15, false),
oz5 = new ObjetZork("oz5", 11, true),
oz6 = new ObjetZork("oz6", 2, false);
ObjetZork[] tabOz = {oz1, oz2, oz3, oz4, oz5, oz6};
// a completer
Iterator<ObjetZork> iter = new FilterArrayIterator(???);
while (iter.hasNext()) {
    System.out.print(": " + iter.next().getNom());
}
System.out.println();
// Affichage: :oz2:oz4:oz5

```

Exercice 3 En Java, `Predicate<E>` est une *functional interface*... profitons en !

Depuis la version 8 (aussi appelée 1.8) du langage, la spécification du langage Java a introduit la notion de *functional interface* qu'elle définit ainsi :

A functional interface is an interface that [...] has just one abstract method (aside from the methods of Object), and thus represents a single function contract.

Les interfaces `Comparator` et `Predicate` sont donc des *functional interface*.

Cette propriété a comme principal avantage qu'elle permet au développeur d'utiliser une *lambda expression* plutôt que de créer explicitement une nouvelle classe à chaque fois qu'il a besoin d'utiliser une nouvelle classe implémentant une *functional interface*. Ainsi dans le cas de l'exemple vu en cours à propos du comparateur utilisé pour trier une liste d'`ObjetZork` selon le poids, l'utilisation d'une *lambda expression* nous permet de remplacer le code suivant (ainsi que de nous éviter la peine de définir la classe `CompareWeight`) :

```

List<ObjetZork> loz = ...
loz.sort(new CompareWeight());
// liste loz triee !

```

peut-être remplacé par le code équivalent :

```

List<ObjetZork> loz = ...
loz.sort((o1, o2) -> o1.getWeight() - o2.getWeight());
// liste loz triee !

```

où `(o1, o2) -> o1.getWeight() - o2.getWeight()` est ce que l'on appelle une *lambda expression* en Java.

1. Dans le code suivant, utilisez une *lambda expression* pour filtrer uniquement les `ObjetZork` de poids strictement inférieur à 15 :

```

ObjetZork oz1 = new ObjetZork("oz1", 5, true),
    oz2 = new ObjetZork("oz2", 45, true),
    oz3 = new ObjetZork("oz3", 7, false),
    oz4 = new ObjetZork("oz4", 15, false),
    oz5 = new ObjetZork("oz5", 11, true),
    oz6 = new ObjetZork("oz6", 2, false);
ObjetZork[] tabOz = {oz1, oz2, oz3, oz4, oz5, oz6};
// a completer
Iterator<ObjetZork> iter = new FilterArrayIterator(???);
while (iter.hasNext()) {
    System.out.print(": " + iter.next().getNom());
}
System.out.println();
// Affichage: :oz1:oz3:oz5:oz6

```

2. Quelle *lambda expression* utiliseriez vous pour filtrer les `ObjetZork` satisfaisant simultanément les trois conditions suivantes :
 - (a) l'`ObjetZork` est non null
 - (b) l'`ObjetZork` est transportable
 - (c) le poids de l'`ObjetZork` est strictement inférieur à 15
3. Compléter le code suivant en utilisant une *lambda expression* pour trier cette liste de `String` en fonction de leur longueur (la longueur d'une instance de `String` peut être obtenue à l'aide de la méthode `public int length()`):

```

List<String> lStr = Arrays.asList("abcd", "aa", "123456",
                                "z", "bb", "cc", "");
lStr.sort(????);
for (String s: lStr) {
    System.out.print(": " + s);
}
System.out.println();
// Affichage: ::z:aa:bb:cc:abcd:123456

```